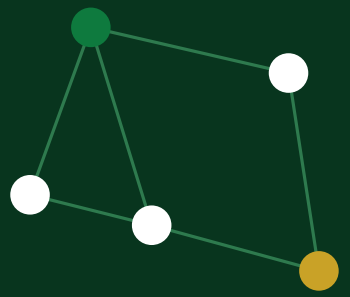
emsi

École Marocaine des Sciences de l'Ingénieur



CHAPITRE 2

Fondements de l'intelligence artificielle

Recherche, heuristiques, métaheuristiques, apprentissage automatique et réseaux de neurones

OBJECTIFS PÉDAGOGIQUES

À la fin de ce chapitre, vous saurez…

1. **Modéliser un problème**
   le représenter comme un espace d'états (états, actions, état initial, but).

2. **Dérouler BFS et DFS**
   simuler pas à pas les listes OPEN / CLOSED et comparer leurs propriétés.

3. **Appliquer GBFS et A***
   utiliser une heuristique, calculer f(n) = g(n) + h(n) et juger l'optimalité.

4. **Concevoir un algorithme génétique**
   sélection, croisement, mutation et critères d'arrêt sur le TSP.

5. **Distinguer ML et Deep Learning**
   supervisé / non supervisé, perceptron et réseaux de neurones.

PLAN

Les six sections du chapitre

1

Espace de recherche

États, actions, graphe et arbre

2

Recherche aveugle

BFS, DFS, coût uniforme

3

Recherche heuristique

Heuristiques, GBFS, A*

4

Métaheuristiques

Algorithme génétique, TSP

5

Machine Learning

Supervisé, non supervisé

6

Deep Learning

Neurone, perceptron, réseaux

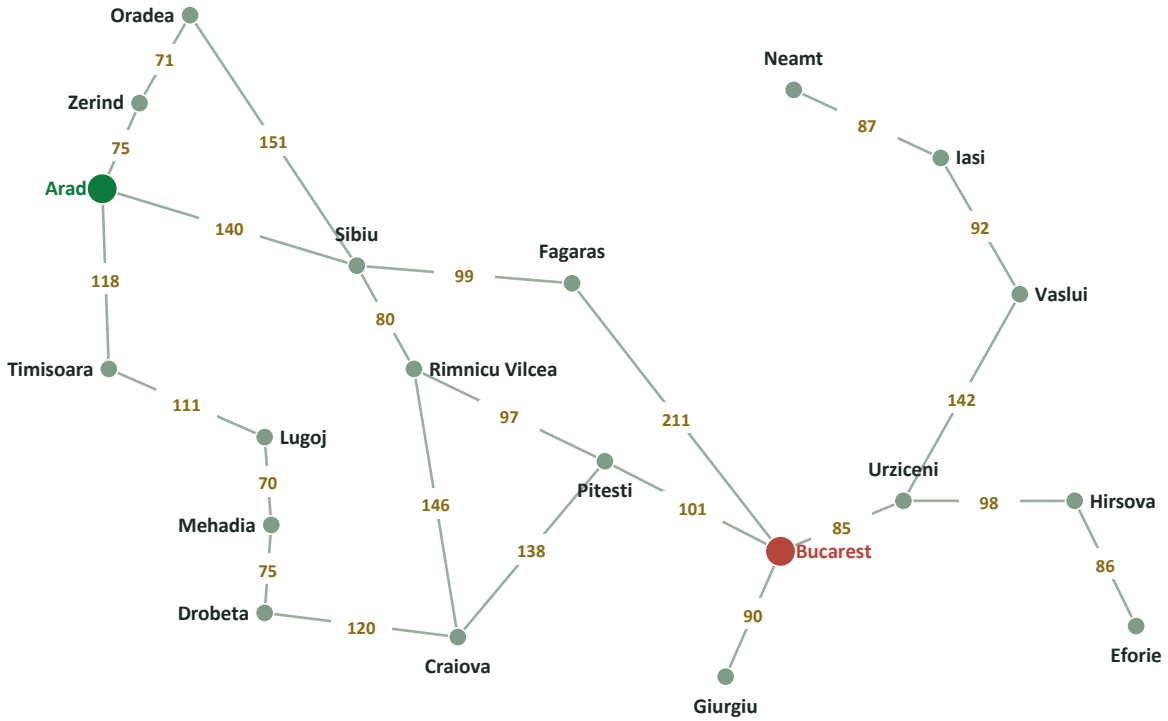
Des travaux dirigés (TD) ponctuent les sections 2, 3 et 4.

SECTION 1

L'espace de recherche

« Comment aller d'Arad à Bucarest ? »

Un problème concret : la carte de Roumanie



La situation

- **Départ :** Arad
- **Arrivée :** Bucarest
- 20 villes reliées par des routes
- Chaque route porte sa distance en km
- **Question :** quel itinéraire choisir ?

SECTION 1 · ESPACE DE RECHERCHE

Qu'est-ce qu'un espace de recherche ?

Pour résoudre un problème avec l'IA, il faut d'abord le **représenter de manière formelle**. On le modélise par un **espace de recherche** (ou espace d'états).

Espace de recherche

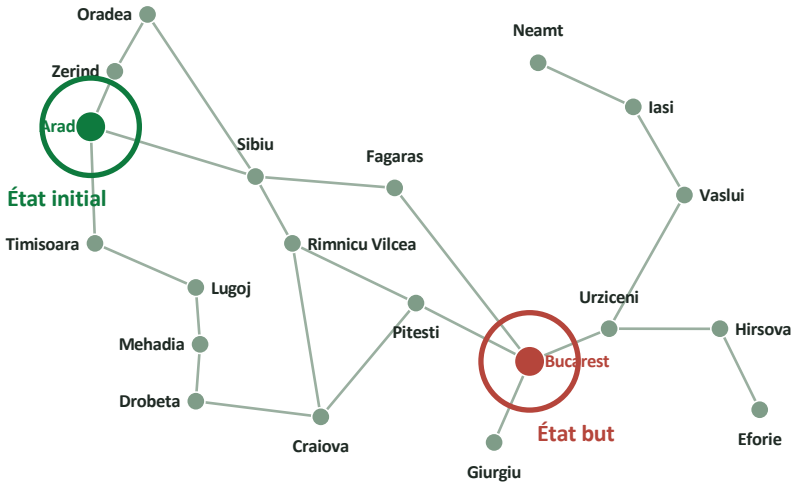
l'ensemble de tous les états possibles d'un problème et des chemins qui les relient.

État

une situation du problème (ex. « être à Sibiu »).

Action

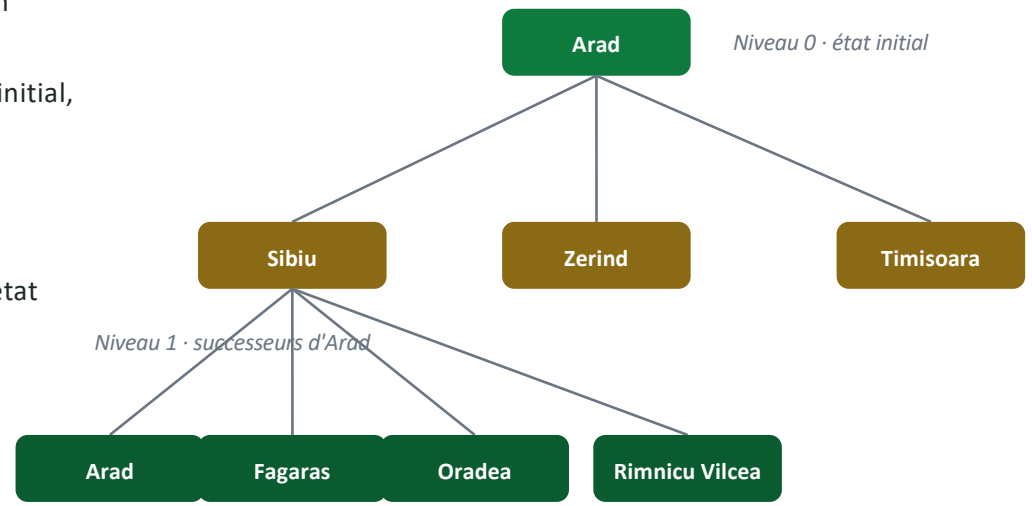
permet de passer d'un état à un autre (ex. « prendre la route vers Fagaras »).



SECTION 1 · ESPACE DE RECHERCHE

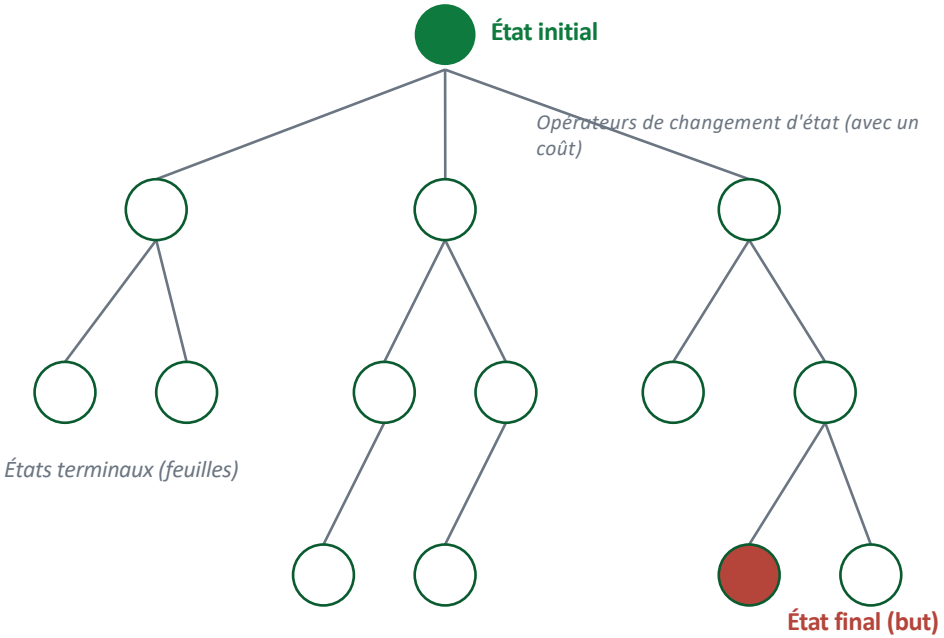
Du graphe à l'arbre de recherche

- L'ensemble des états et des actions se représente par un graphe ou un arbre de recherche.
- Traverser cet arbre = explorer les états à partir de l'état initial, en suivant les actions possibles.
- **Découvert :** état atteint pour la première fois.
- **Traité :** toutes ses actions ont été examinées.
- **But :** trouver un chemin solution reliant l'état initial à l'état but.



Remarque : Arad réapparaît comme successeur de Sibiu — d'où l'utilité de mémoriser les états déjà traités (CLOSED).

Les structures de la recherche : OPEN et CLOSED



OPEN

Frontière : nœuds découverts mais pas encore développés. Sa structure dépend de l'algorithme (file, pile, file de priorité).

CLOSED

Nœuds déjà développés (visités), mémorisés pour éviter les re-visites et les boucles.

SOLUTION

Un chemin : la séquence d'états qui mène de l'état initial à l'état final.

SECTION 1 · ESPACE DE RECHERCHE

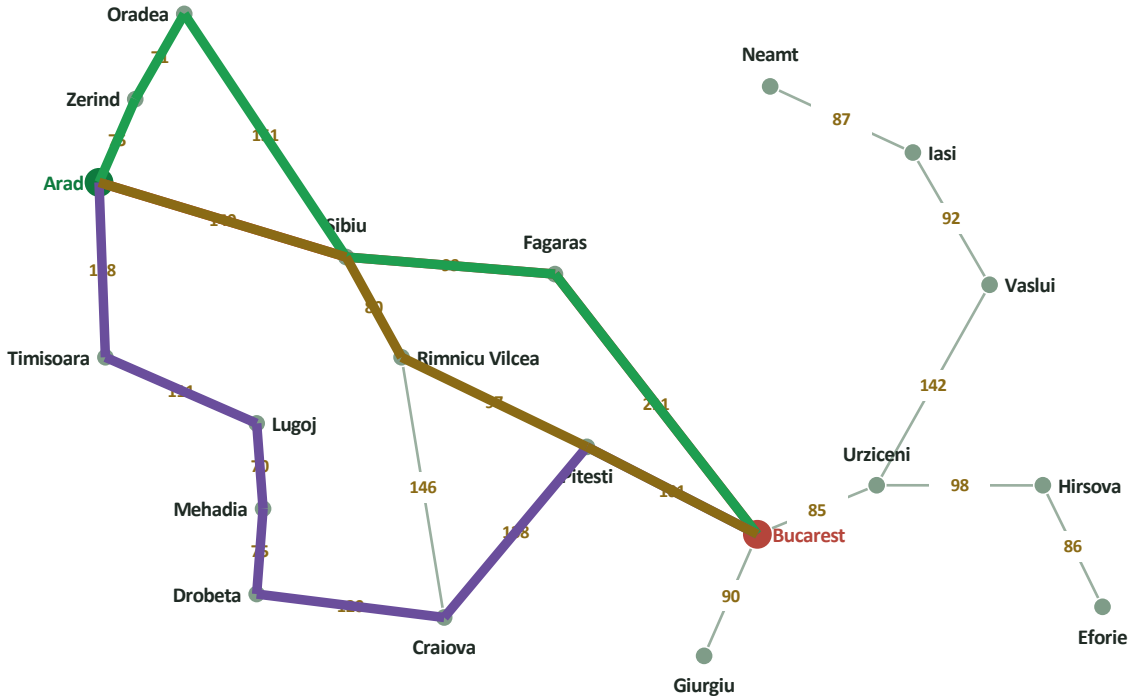
Le processus général de recherche

Initialiser : OPEN = {état initial}, CLOSED = ∅, parent[·] = ∅ puis répéter les étapes 1 à 5 :

1. **Choisir —** dans OPEN le nœud n selon une stratégie : le plus ancien, le plus récent, le plus « prometteur », le moins coûteux…
2. **Tester le but —** si n est l'état but → reconstruire le chemin via parent[·] et s'arrêter.
3. **Développer n —** appliquer les opérateurs pour générer ses successeurs.
4. **Enregistrer —** chaque successeur v non déjà vu : parent[v] = n, puis l'ajouter à OPEN.
5. **Déplacer —** n de OPEN vers CLOSED, puis recommencer.

Toute la différence entre BFS, DFS, GBFS et A tient à l'étape 1 : la stratégie de choix dans OPEN.*

Plusieurs chemins solutions possibles



Chemin 1

140 + 99 + 211 = 450 km

Chemin 2

75 + 71 + 151 + 99 + 211 = 607 km

Chemin 3

118 + 111 + 70 + 75 + 120 + 138 + 101 = 733 km

Chemin 4

140 + 80 + 97 + 101 = 418 km

Le plus court : 418 km. Comment un algorithme le trouve-t-il ?

QUIZ · À VOUS DE JOUER

Pour aller d'Arad à Bucarest, quel est l'état initial, l'état but, et que représente une action ?

- Initial : Arad
- But : Bucarest
- Action : emprunter une route
- Action : choisir une ville au hasard

Réponse État initial = Arad, état but = Bucarest ; une action = emprunter une route vers une ville voisine (avec un coût en km).

2

SECTION 2

Les algorithmes de recherche aveugle

« Explorer sans boussole »

SECTION 2 · RECHERCHE AVEUGLE

Trois grandes familles d'algorithmes de recherche

1

Algorithmes aveugles

Explorent l'espace d'états sans information supplémentaire sur le problème.

BFS · DFS · coût uniforme

2

Algorithmes informés (heuristiques)

Utilisent une estimation h(n) de la distance au but pour guider l'exploration.

GBFS · A*

3

Algorithmes métaheuristiques

Font évoluer une population de solutions pour approcher l'optimum des problèmes trop vastes.

Algorithme génétique · recuit simulé…

Plus d'information sur le problème → exploration plus ciblée

SECTION 2 · RECHERCHE AVEUGLE

Les algorithmes aveugles (non informés)

Ils explorent l'espace d'états sans information supplémentaire : seule la structure du graphe est connue.

Recherche en largeur (BFS)

Développe les nœuds niveau par niveau.

Structure : file FIFO

Recherche en profondeur (DFS)

Descend le plus loin possible, puis revient en arrière.

Structure : pile LIFO

Recherche à coût uniforme (UCS)

Développe d'abord le nœud de plus faible coût cumulé g(n).

Structure : file de priorité

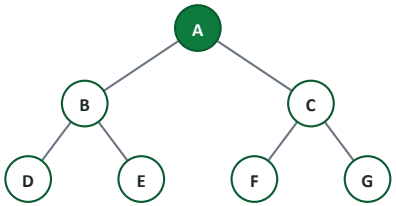
Le coût uniforme (proche de l'algorithme de Dijkstra) garantit le chemin de coût minimal ; sur la carte de Roumanie, il retrouve les 418 km.

SECTION 2 · RECHERCHE EN LARGEUR (BFS)

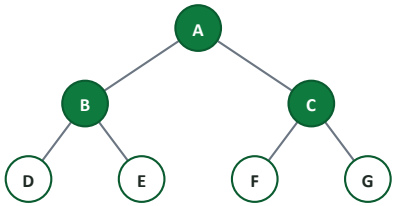
Principe : explorer niveau par niveau

La recherche en largeur (*Breadth-First Search*) développe les nœuds par niveau. On part de l'état initial, puis on ajoute à la frontière tous les états obtenus par les actions possibles. Ces états sont stockés dans une **file FIFO** (First In, First Out) : les premiers insérés sont les premiers explorés.

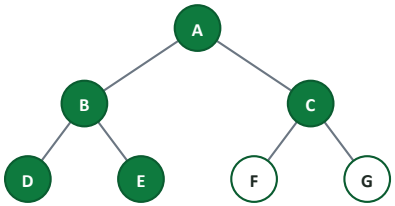
- On développe tous les nœuds du niveau i avant ceux du niveau i + 1.
- Les nouveaux successeurs sont placés à la fin de la file.



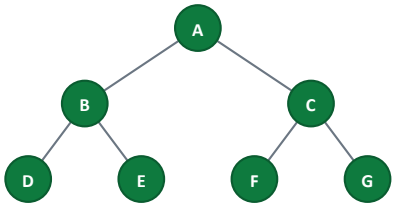
Niveau 0



Niveau 1



Niveau 2 (début)

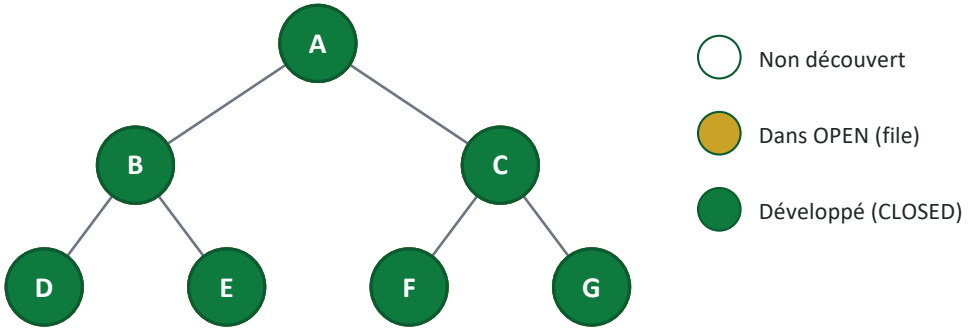


Niveau 2

Ordre de visite : A – B – C – D – E – F – G

SECTION 2 · RECHERCHE EN LARGEUR (BFS)

Déroulé pas à pas : OPEN est une file



Étape	Nœud développé	OPEN (file)	CLOSED (ordre de visite)
0	—	[A]	∅
1	**A**	[B, C]	A
2	**B**	[C, D, E]	A B
3	**C**	[D, E, F, G]	A B C
4	**D**	[E, F, G]	A B C D
5	**E**	[F, G]	A B C D E
6	**F**	[G]	A B C D E F
7	**G**	[ ]	A B C D E F G

```python
from collections import deque

def bfs(graph, start):
    # File (FIFO) des états à explorer
    queue = deque([start])
    # États déjà découverts
    visited = set([start])

    while queue:
        # Retirer le premier élément de la file
        node = queue.popleft()
        print("Visite :", node)

        # Parcourir les voisins (successeurs)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Avantages et inconvénients du BFS

✓ Avantages

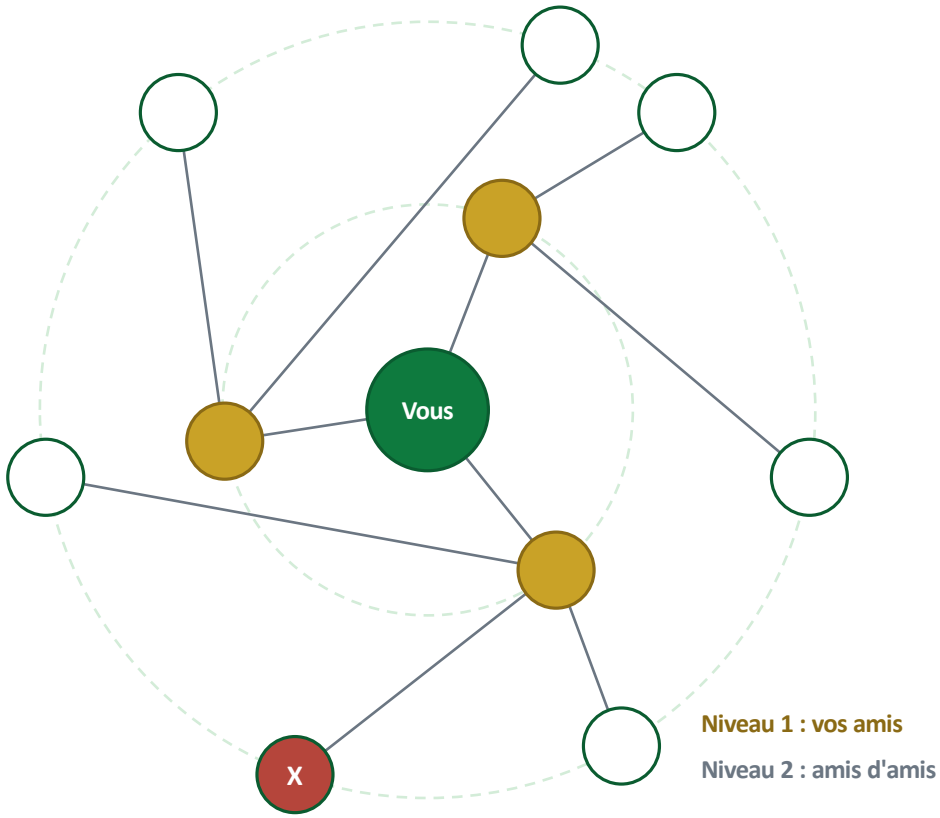
- **Trouve toujours la solution la plus courte** (en nombre d'actions).
- **Exploration systématique** (niveau par niveau).
- Facile à comprendre et à implémenter.
- **File FIFO** → ordre d'exploration garanti.

✗ Inconvénients

- **Mémoire importante :** il faut stocker tous les nœuds d'un niveau avant de passer au suivant.
- **Temps élevé** si l'espace de recherche est très grand.
- Peu adapté quand la solution est **profonde mais unique.**

Exemple : calculer la distance sociale

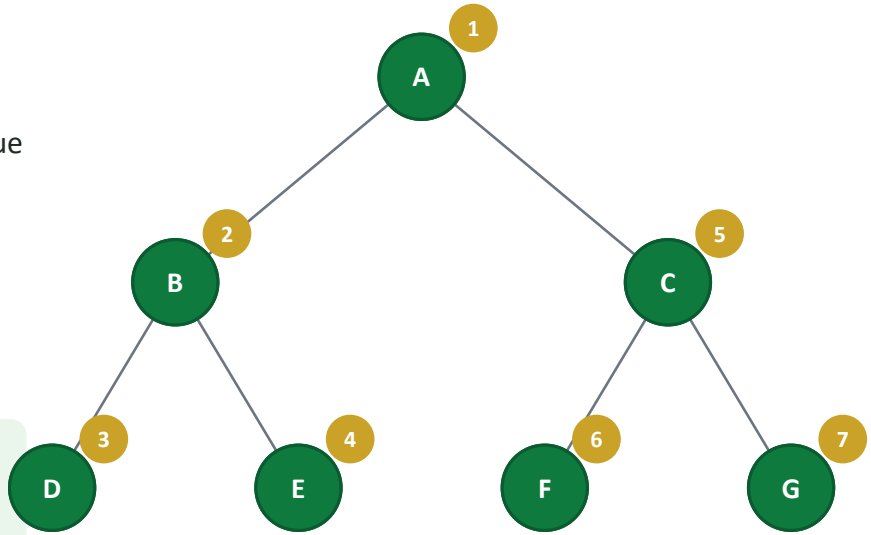
- Sur LinkedIn ou Facebook, on peut lire « vous êtes à 2 connexions de X ».
- Pour calculer ce **degré de séparation**, l'algorithme part de vous et explore **niveau par niveau** : vos amis, puis les amis de vos amis, etc.
- Le premier niveau où X apparaît donne la distance minimale : c'est exactement le BFS.



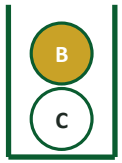
X est à 2 connexions de vous.

Principe : descendre le plus loin possible

La recherche en profondeur (*Depth-First Search*) descend le plus loin possible dans l'arbre avant de revenir en arrière. On choisit un successeur et l'on continue jusqu'à un nœud sans successeur ou déjà visité ; l'algorithme effectue alors un **retour arrière (backtracking)** pour explorer d'autres branches.



Pile LIFO (Last In, First Out) : le dernier état inséré est le premier exploré. Les nouveaux états vont au sommet de la pile.

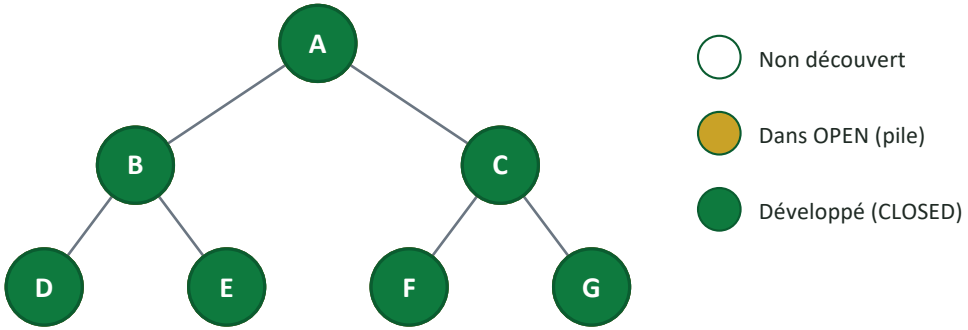


← *sommet : sorti en premier*

Ordre de visite : A – B – D – E – C – F – G

SECTION 2 · RECHERCHE EN PROFONDEUR (DFS)

Déroulé pas à pas : OPEN est une pile



Étape	Nœud développé	OPEN (pile)	CLOSED (ordre de visite)
0	—	[A]	∅
1	**A**	[C, B]	A
2	**B**	[C, E, D]	A B
3	**D**	[C, E]	A B D
4	**E**	[C]	A B D E
5	**C**	[G, F]	A B D E C
6	**F**	[G]	A B D E C F
7	**G**	[ ]	A B D E C F G

```python
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()

    # Marquer le nœud comme visité
    visited.add(start)
    print("Visite :", start)

    # Explorer les voisins un par un (en profondeur)
    for neighbor in graph[start]:
        if neighbor not in visited:
            print(f"-> {start} vers {neighbor}")
            dfs(graph, neighbor, visited)
            # Backtracking (retour arrière)
            print(f"<- retour à {start}")
```

SECTION 2 · RECHERCHE EN PROFONDEUR (DFS)

Avantages et inconvénients du DFS

✓ Avantages

- **Peu de mémoire :** seule la pile des nœuds en cours est conservée.
- **Exploration rapide en profondeur :** atteint vite un état profond.
- **Implémentation simple** avec la récursion.
- **Utile pour un parcours complet** (ex. détection de cycles).

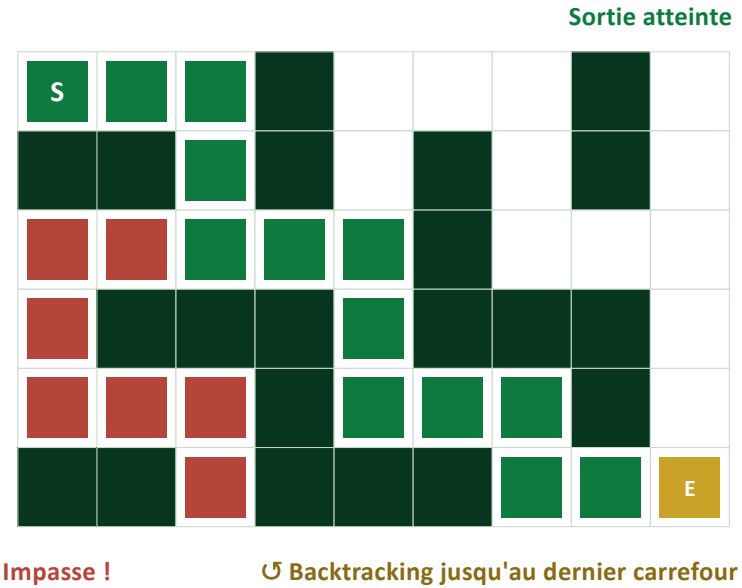
✗ Inconvénients

- **Ne garantit pas la solution la plus courte** (contrairement au BFS).
- Peut s'enfoncer dans une branche inutile → **perte de temps.**
- **Risque de boucle infinie** si le graphe a des cycles et qu'on ne mémorise pas les visités.
- Moins adapté quand la solution est proche de la racine.

SECTION 2 · DFS EN PRATIQUE

Exemple : un robot explore un labyrinthe

- Un robot doit trouver la sortie d'un grand labyrinthe.
- Le DFS choisit un chemin et avance en profondeur jusqu'à la sortie… ou une impasse.
- Si le chemin est bloqué : **backtracking** (retour arrière) pour tester une autre possibilité.
- **Efficace :** il explore tout le labyrinthe avec peu de mémoire, mais ne garantit pas le chemin le plus court.



SECTION 2 · SYNTHÈSE

BFS ou DFS : comment choisir ?

Critère	BFS (largeur)	DFS (profondeur)
Structure de OPEN	File FIFO	Pile LIFO (ou récursion)
Ordre d'exploration	Niveau par niveau	Branche par branche
Solution la plus courte (en actions)	Oui (garanti)	Non
Mémoire	Élevée : O(b^d)	Faible : O(b·m)
Risque	Explosion mémoire	Boucles, branches inutiles
Usage typique	Plus court chemin, réseaux sociaux	Labyrinthes, parcours complet, cycles

b = facteur de branchement · d = profondeur de la solution · m = profondeur maximale de l'arbre

QUIZ · FILE OU PILE ?

Quel algorithme utilise une pile (LIFO) et ne garantit pas le chemin le plus court ?

- BFS
- **DFS**
- Coût uniforme

Réponse DFS : il dépile toujours le dernier nœud ajouté et peut trouver un chemin plus long que le BFS.

TD

TRAVAUX DIRIGÉS 1

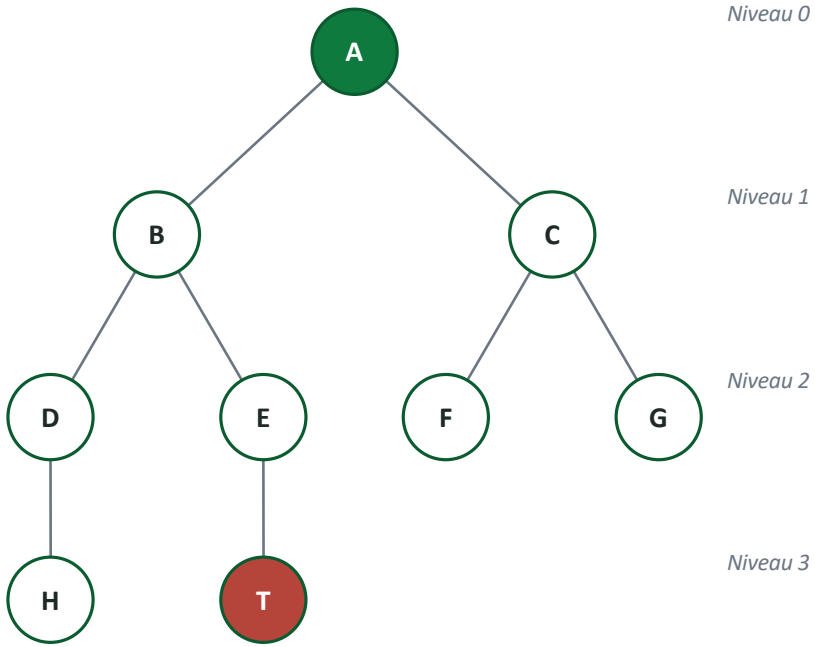
Recherche aveugle : BFS et DFS

Simuler, comparer, justifier

TRAVAUX DIRIGÉS 1 · EXERCICE 1

Trouver la cible T dans un arbre

- A est la racine ; A a deux enfants : B (à gauche) et C (à droite).
- B a deux enfants : D et E ; C a deux enfants : F et G.
- D a un enfant : H ; E a un enfant : T (la cible). F et G n'ont pas d'enfants.
- **Règle de visite :** toujours explorer de la gauche vers la droite.

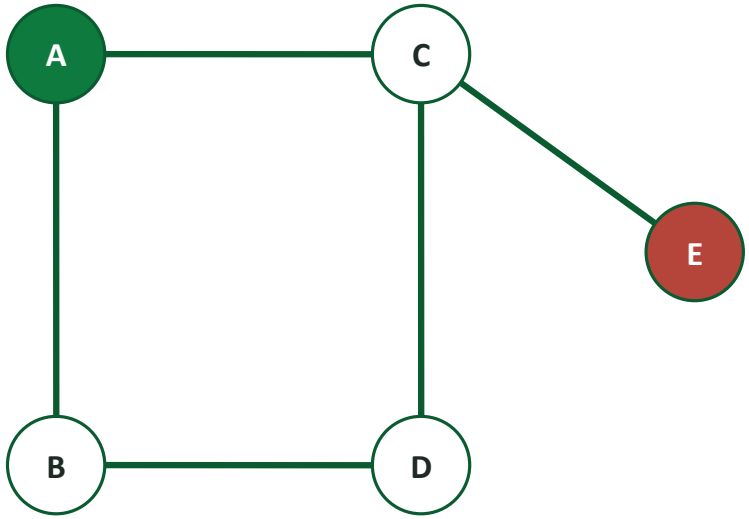


- **BFS :** ordre des nœuds visités ? chemin le plus court (en liens) de A à T ?
- **DFS :** ordre des nœuds visités ? chemin trouvé ?
- **Analyse :** pourquoi le BFS garantit-il l'optimalité, et pourquoi le DFS peut-il aboutir à un chemin plus long ?

TRAVAUX DIRIGÉS 1 · EXERCICE 2

Gestion du trafic : du carrefour A au carrefour E

- Chaque lettre est un carrefour, chaque ligne une route.
- **Départ :** A · **Cible :** E
- **Règle :** explorer les voisins par ordre alphabétique (B avant C, D avant E…).
- Routes : A–B, A–C, B–D, C–D, C–E.

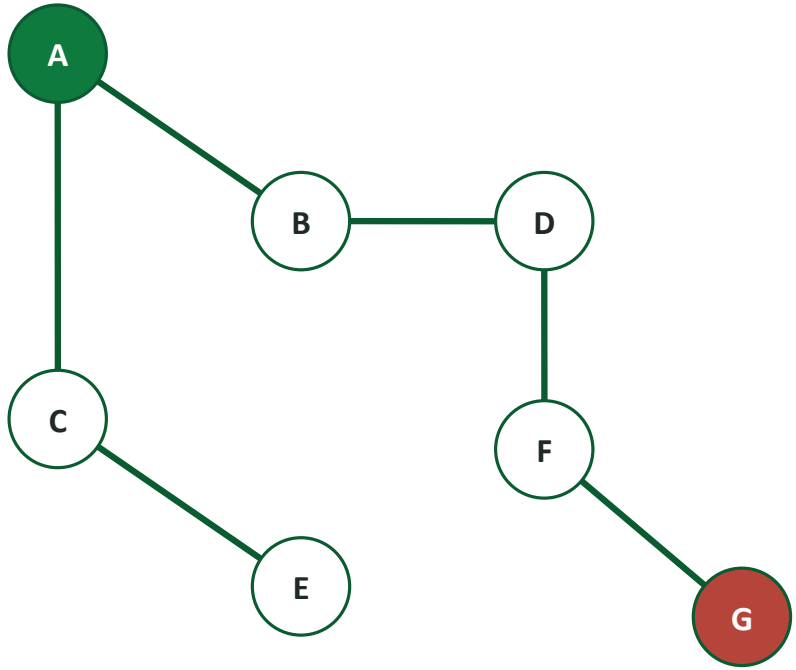


- **BFS :** séquence des carrefours visités jusqu'à E ? nombre minimal de changements de route de A à E ?
- **DFS :** séquence visitée jusqu'à E ? chemin trouvé ?
- **Analyse :** pourquoi un service comme Google Maps privilégie-t-il le BFS (ou ses variantes pondérées) ? Dans quel cas utiliser le DFS ici ?

TRAVAUX DIRIGÉS 1 · EXERCICE 3

Du carrefour A au carrefour G

- **Départ :** A · **Cible :** G · ordre alphabétique
- Routes : A–B, A–C, B–D, C–E, D–F, F–G.
- E est une impasse (feuille).



- Simulez le BFS : listez les carrefours visités et donnez le chemin trouvé.
- Simulez le DFS : même question.
- Comparez le nombre de nœuds visités par chaque algorithme. Lequel est le plus économe ici, et pourquoi ?